

Urban Mobility Data Explorer

Technical Report - Team 2 | ALU Enterprise Web Development

June 2025

Nshimyumurwa Mary Therese (Team Lead) | Clive Mushipe (Data Pipeline) | Davy Dushimiyimana (Frontend Dashboard) |
Aimable Bancunguye (DSA Implementation) | Eloi Mizero (Documentation and Technical Report)

1. Problem Framing

NYC yellow taxi trips produce millions of records annually. Raw TLC logs include pickup/dropoff times, fares, distances, and zone IDs, but are hard to interpret without cleaning, enrichment, and a usable interface.

We built the Urban Mobility Data Explorer: a full-stack app that processes NYC TLC yellow taxi data, stores it in SQLite, and serves it via a REST API to an interactive dashboard. Users filter trips by borough, time of day, fare, and distance; view KPIs; and inspect individual records.

Data: `yellow_tripdata` parquet files, `taxi_zone_lookup.csv`, `taxi_zones` spatial metadata (<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>).

Key questions: Which boroughs/zones drive demand? How do fares, duration, and speed vary by time of day? What data quality issues exist and how are exclusions logged?

[TBD - Team] Final scope once January 2019 data is fully loaded.

2. System Architecture

Three-tier design: data layer, API layer, presentation layer.

Data flow: TLC parquet -> `data_pipeline.py` (clean + derive) -> SQLite (`mobility.db`) -> Flask REST API -> Chart.js dashboard in the browser.

Data layer (Clive / database/): `schema.sql` defines zones (265 NYC zones), trips (cleaned records + derived features), and `exclusion_log` (audit trail). `data_pipeline.py` loads parquet from `data/`, cleans outliers, derives `trip_duration_minutes`, `fare_per_mile`, and `speed_mph`, then writes to `mobility.db`. Cleaning removes nulls, duplicates, fares outside \$0-\$500, distances over 100 mi, invalid passenger counts, and trips where dropoff precedes pickup or duration exceeds 6 h.

API layer (Therese / backend/): Flask app (`app.py`) on port 5000 with CORS. Endpoints include `/api/summary`, `/api/trips` (paginated filters), `/api/summary/by-borough`, `/api/summary/by-hour`, `/api/zones`, `/api/exclusions`, and `/api/health`. `db.py` handles SQLite; `seed.py` provides synthetic fallback data.

Presentation layer (Davy / frontend/): Chart.js dashboard with KPI meters, demand charts, borough revenue, time-of-day breakdown, zone rankings, traffic efficiency scatter (speed vs fare/mile), exclusion log, and paginated trip explorer. Filters support borough, time-of-day, date range, passenger count, fare and distance bounds. Trip table supports column sorting and row-level detail modal.

[TBD - Davy] JS module details. [TBD - Therese] Additional API decisions.

3. Algorithm Explanation

Custom DSA: Insertion Sort (Aimable / dsa/algorithm.py)

Problem: Rank trips by fare to find the most/least expensive across boroughs without using `sort()` or `sorted()`.

How it works: For each index i , hold the current trip and shift larger-fare neighbours right until the correct position is found, then insert. Operates in-place on a list of trip dicts keyed by `fare_amount`.

Pseudo-code: for i in $1..n-1$: $j = i-1$; while $j \geq 0$ and `trips[j].fare > current.fare`: shift right; insert current at $j+1$.

Complexity: Time $O(n)$ best, $O(n^2)$ average/worst. Space $O(1)$.

Exports: `insertion_sort_by_fare()`, `get_top_expensive_trips(n)`, `get_cheapest_trips(n)`, `benchmark()`. Fetches up to 500 DB rows (no SQL ORDER BY) and sorts in Python.

[TBD - Aimable] Benchmark results, algorithm choice rationale, API integration plans.

4. Insights

Preliminary findings (README; pending full-dataset verification):

- Manhattan origin for 90%+ of trips (CBD concentration).
- Higher avg fares midnight-3am vs daytime.
- speed_mph drops during morning rush (7-9am).

Dashboard charts surface these: hourly demand, revenue by borough, time-of-day split, traffic efficiency scatter, borough rankings.

[TBD - All] Exact numbers and chart screenshots. [TBD - Clive] Exclusion log totals and top removal reasons.

5. Reflection

Therese: [TBD] API design, pagination, query performance on large trip tables.

Clive: [TBD] Cleaning threshold trade-offs (\$500 fare cap, 100 mi distance cap) and exclusion_log transparency.

Davy: [TBD] Filter UX, chart choices, loading states and error handling.

Aimable: [TBD] Manual sort vs built-in sort; when $O(n^2)$ is acceptable.

Eloi: Early report structure tracks real decisions; docs/project_notes.md captures team input as work progresses rather than generic filler.

[TBD - Team] What we would add with more time: borough map using taxi_zones shapefiles, additional months of TLC data, and a /api/trips/ranked endpoint powered by the DSA module.